Suggested Answer

Howard Smith

2023-03-10

Preliminary remarks

Note that this answer is not intended to be a perfect answer and can be improved upon. However it should give you an idea of the sort of discussion we are looking for in the Empirical IO question. I found that R-markdown to PDF answers were convenient to mark so I would ask you submit such an output.

```
rm(list = ls())
set.seed(1042018)
```

```
library(BLPestimatoR)
library(lmtest)
```

```
## Loading required package: zoo

##
## Attaching package: 'zoo'

## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```
library(sandwich)
library(modelsummary)
library(ivreg)
library(nleqslv)
library(readr)
```

Question 1: Simple Logit model

Part a)

Since there isn't a product name or an mpd variable in the library dataset, we add them in from the dataset on the course website. (Points to an address in my Dropbox).

```
autoData <- read_csv("C:/Users/Howard Smith/Dropbox/AdERM/Data/autoData.csv")
```

```
## Rows: 2217 Columns: 18
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr  (1): model_id
## dbl (17): car_id, firm_id, price, domestic, japan, european, const, hpwt, ai...
##
## i Use `spec()` to retrieve the full column specification for this data.
```

```
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
mpd<-autoData$mpd
product_name<-autoData$model_id
logit_depvar<-autoData$logit_depvar

# Seems to be sorted the same way so can append:
productData_cars <- cbind(productData_cars, mpd, product_name, logit_depvar)

#head(productData_cars)


# Add owner matix (in dummies_cars) to productData
#head(dummies_cars)
own_pre <- dummies_cars
colnames(own_pre) <- paste0("company", 1:26)
productData_cars <- cbind(productData_cars, own_pre)
```

In order to estimate logit by OLS we inverted the market share function following Berry (1994) to obtain the following linear regression form:

$$\ln s_{jt} - \ln s_{0t} = \alpha + \beta x_{jt} + \xi_{jt}$$

```r
model_ols <- lm(logit_depvar~hpwt+air+mpd+space+price, data=productData_cars)
summary(model_ols)
```

```
##
## Call:
## lm(formula = logit_depvar ~ hpwt + air + mpd + space + price,
##     data = productData_cars)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -6.3473 -0.6540  0.1371  0.7147  2.4568
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -10.068854   0.252879 -39.817  < 2e-16 ***
## hpwt         -0.120558   0.277234  -0.435    0.664
## air          -0.034745   0.072806  -0.477    0.633
## mpd           0.263170   0.043118   6.104 1.22e-09 ***
## space         2.341023   0.125180  18.701  < 2e-16 ***
## price        -0.088731   0.004026 -22.041  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.083 on 2211 degrees of freedom
## Multiple R-squared:  0.3874, Adjusted R-squared:  0.386
## F-statistic: 279.6 on 5 and 2211 DF,  p-value: < 2.2e-16
```

```r
# Standard errors clustered on products:

ols1_clust<- coeftest(model_ols,  vcov = vcovCL, type = "HC1", cluster = ~ product_name)
```

	Baseline	Clustered
(Intercept)	−10.069***	−10.069***
	(0.253)	(0.369)
hpwt	−0.121	−0.121
	(0.277)	(0.377)
air	−0.035	−0.035
	(0.073)	(0.103)
mpd	0.263***	0.263***
	(0.043)	(0.059)
space	2.341***	2.341***
	(0.125)	(0.184)
price	−0.089***	−0.089***
	(0.004)	(0.006)
Num.Obs.	2217	2217
R2	0.387	
F	279.635	
RMSE	1.08	

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

```
modelsummary(list("Baseline "=model_ols, "Clustered"=ols1_clust),
             stars = TRUE,
             fmt = 3,
             gof_omit = 'Log.Lik|R2 Adj.|AIC|BIC')
```

```
## Warning in !is.null(rmarkdown::metadata$output) && rmarkdown::metadata$output
## %in% : 'length(x) = 2 > 1' in coercion to 'logical(1)'
```

```
# Clustering by model_id
```

We cluster on products which allows us to relax the assumption that the ξ term is iid across markets for the same product.

The coefficient on price is the right sign (negative) and it is significant. But it is to be expected that prices are endogenous since a price-setter is likely to be aware of the unobserved market-level quality-shifter in the ξ term.

Part b)

```
# Create a function that generates the IVs
nobs <- nrow(productData_cars)

#Make a data frame with the 5 characteristics
X <- data.frame(productData_cars$const, productData_cars$hpwt,
                productData_cars$air, productData_cars$mpg, productData_cars$space)

#Prepare two nobs x 5 matrices
sum_other <- matrix(NA, nobs, ncol(X))
sum_rival <- matrix(NA, nobs, ncol(X))

#Loop over observations adding characteristics within the same year:

for (i in 1:nobs) {
```

```r
  other_ind <- productData_cars$firmid == productData_cars$firmid[i] &
    productData_cars$cdid == productData_cars$cdid[i] &
    productData_cars$id != productData_cars$id[i]
  rival_ind <- productData_cars$firmid != productData_cars$firmid[i] &
    productData_cars$cdid == productData_cars$cdid[i]
  sum_other[i, ] <- colSums(X[other_ind == 1, ])
  sum_rival[i, ] <- colSums(X[rival_ind == 1, ])
}

#Give them column names:
colnames(sum_other) <- paste0("IV", 1:5)
colnames(sum_rival) <- paste0("IV", 6:10)

#head(sum_other)
#head(sum_rival)
#Add them to the data frame
productData_cars <- cbind(productData_cars, sum_other, sum_rival)

#head(productData_cars)
```

```r
# 2SLS model
model_iv <- ivreg(logit_depvar ~ 0 + const + hpwt + air + mpd + space|
                    price|
                    0 + const + hpwt + air + mpd + space + IV1 + IV2 + IV3 + IV4 + IV5 +
                       IV6+ IV7+ IV8 + IV9 + IV10,
                    data=productData_cars)
summary(model_iv)
```

```
##
## Call:
## ivreg(formula = logit_depvar ~ 0 + const + hpwt + air + mpd +
##     space | price | 0 + const + hpwt + air + mpd + space + IV1 +
##     IV2 + IV3 + IV4 + IV5 + IV6 + IV7 + IV8 + IV9 + IV10, data = productData_cars)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.54372 -0.64253  0.09682  0.71254  4.24106
##
## Coefficients:
##       Estimate Std. Error t value Pr(>|t|)
## price -0.13634    0.01043 -13.070  < 2e-16 ***
## const -9.91082    0.26268 -37.729  < 2e-16 ***
## hpwt   1.24503    0.39632   3.141 0.001703 **
## air    0.49183    0.12977   3.790 0.000155 ***
## mpd    0.16865    0.04835   3.488 0.000496 ***
## space  2.28996    0.12949  17.685  < 2e-16 ***
##
## Diagnostic tests:
##                   df1  df2 statistic p-value
## Weak instruments   10 2202     41.43 < 2e-16 ***
## Wu-Hausman          1 2210     26.62 2.7e-07 ***
## Sargan              9   NA    257.36 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

	Clustered OLS	Clustered IV
(Intercept)	−10.069***	
	(0.369)	
hpwt	−0.121	1.245*
	(0.377)	(0.536)
air	−0.035	0.492**
	(0.103)	(0.187)
mpd	0.263***	0.169*
	(0.059)	(0.067)
space	2.341***	2.290***
	(0.184)	(0.187)
price	−0.089***	−0.136***
	(0.006)	(0.016)
const		−9.911***
		(0.378)
Num.Obs.	2217	2217

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

```
##
## Residual standard error: 1.116 on 2211 degrees of freedom
## Multiple R-Squared: 0.9789,  Adjusted R-squared: 0.9789
## Wald test: 1.708e+04 on 6 and 2211 DF,  p-value: < 2.2e-16
```

```r
# Get clustered SEs

model_iv_clust <- coeftest(model_iv, vcov = vcovCL, type = "HC1", cluster = ~ product_name)

 modelsummary(list("Clustered OLS"=ols1_clust, "Clustered IV"=model_iv_clust),
               stars = TRUE,
              fmt = 3,
              gof_omit = 'Log.Lik|R2 Adj.|AIC|BIC')
```

The 'price' coefficient became more negative (higher in magnitude) compared to the OLS estimates. This change is a consequence of using instruments for price. The direction of change is consistent with price being positively correlated with ξ, which is to be expected in most oligopoly price-setting models where the price-setter is aware of ξ.

Part c)

Our goal is to obtain the marginal costs for each product. To do this we use mark-up formula under multi-product Nash pricing assumption. We use the following formula, as derived in the lecture notes

$$p_{jt} - c_{jt} = -\frac{1}{\alpha(1 - s_{ft})}$$

where alpha is the price parameter in the mean utility index, p_{jt} is the price of product j and s_ft is the firm's market share.

```r
# Get firm shares

make_firm_shares <- function(){
```

```
Z <- productData_cars$share

sum_samef <- matrix(NA, nobs, 1)

for (i in 1:nobs) {
  same_ind <- productData_cars$firmid == productData_cars$firmid[i]  &
    productData_cars$cdid == productData_cars$cdid[i]

  sum_samef[i,1] <- sum(Z[same_ind == 1])

}

colnames(sum_samef) <- paste0("sharef", 1)

return(sum_samef)

}

 sf <-make_firm_shares()
```

```
# Get alpha
alpha <- model_iv$coefficients[1]
# Get prices
p = productData_cars$price
# Apply the formula to compute the markup
markup <- - (1/alpha)*(1/(1-sf))
# Obtain the marginal cost
mc <- p - markup
summary(mc)
```

```
##     sharef1
##  Min.   :-3.9419
##  1st Qu.:-0.8099
##  Median : 1.2016
##  Mean   : 4.2797
##  3rd Qu.: 5.5809
##  Max.   :61.2614
```

```
mean(mc<0)
```

```
## [1] 0.3513757
```

We see that 35% of the marginal costs we estimated are negative. This suggests that the estimated model has implausible price-elasticities. We conclude that the price elasticities from the simple logit model are too low in magnitude even when instruments are used to allow for endogenous prices.

Exercise 2: Mixed Logit

Part a)

Generate the draws for income variable using the following parameters for the lognormal distribution.

```
sigma_v = 1.72

# Log income means for years 1971 - 1990:
```

```r
incomeMeans = c(2.01156, 2.06526, 2.07843, 2.05775, 2.02915, 2.05346, 2.06745,
                2.09805, 2.10404, 2.07208, 2.06019, 2.06561, 2.07672, 2.10437,
                2.12608, 2.16426, 2.18071, 2.18856, 2.21250, 2.18377)

# 500 draws, standard normal (can reduce if your code runs slow!):
#set.seed(130)
ndraws = 500
nu_0 <-matrix( rnorm(20*ndraws,mean=0,sd=1), nrow=20, ncol=ndraws)
y_means <-matrix( incomeMeans, nrow=20, ncol=ndraws)

# income draws are log normal
y_it = exp(y_means + sigma_v*nu_0)

# We want the inverse a la BLP (1995)
inv_inc = 1/y_it


# BLPEstimatoR needs us to label columns like this:
colnames(inv_inc) <- paste0("draw_", 1:ndraws)

# BLPEstimatoR needs market identifier (1-20) in first column (& must be labelled such)
inv_inc <- cbind(c(1:20), inv_inc)
colnames(inv_inc)[1] <- "cdid"



# BLPEstimatoR wants demographics as a list in R:
demog_auto <- list(invinc = inv_inc)
```

Part b)

There are 11 parameters to estimate. We now estimate these by GMM using the assumption that ξ_{jt} is mean-independent of the following 15 instruments: (i) the 10 instruments you constructed in question 1 and (ii) the five product characteristics that are in the vector x_{it}, with a BFGS optimizer, custering on product.

```r
# Set up the model using as.formula()

#Set up the model and assign to blps_model
  #The variable to be explained is first and precedes a "~": share~.
  #Then the explanatory variables are in four potentially-overlapping groups

  #share ~ linear variables |  exogenous variables |
  #         variables with random coefficients | instruments

  # when a "0" is included to stop BLPestimatoR adding a constant (its default).
  #         ... not needed as we have put one in explicitly.

blps_model <- as.formula("share ~ 0 + const + hpwt + air + mpd + space |
                                  0 + const + hpwt + air + mpd + space |
                                  0 + price + const + hpwt + air + mpd + space |
                                  0 + IV1 + IV2 + IV3 + IV4 + IV5 + IV6 + IV7 + IV8 + IV9 + IV10")
```

```r
# Create a data object, using BLP_data().
```

```r
# Specify the product, market, cluster, productData, demographicdraws, etc.

car_data <- BLP_data(
  model = blps_model,
  market_identifier = "cdid",
  product_identifier = "id",
  productData = productData_cars,
#  additional_variables = paste0("company", 1:26), # we use this in merger (later)
  group_structure = "product_name",
  blp_inner_tol = 1e-9,                    #tolerance of contraction
  blp_inner_maxit = 5000,                  #max iterations of contraction
  integration_method = "MLHS",             #Modified Latin Hypercube Sampling
  demographic_draws = demog_auto,
  integration_accuracy = ndraws, integration_seed = 48 #ndraws draws for integral
)
```

```
## Mean utility (variable name: `delta`) is initialized with 0 because of missing or invalid par_delta a
```

```r
#set.seed(130)

# We want a (K+1=6) x 2 matrix.
# Arranged so we get zeros where we don't want a sigma parameter
# Like BLP: I want unobserved heterogeneity on the 5 characteristics not price
# ... and I want demographics on price not any other chars:

theta_guesses <- matrix(c(0,1,1,1,1,1,-50, 0,0,0,0,0), nrow=6, ncol=2)

# Naming the rows and columns is needed.
rownames(theta_guesses) <- c("price", "const", "hpwt", "air", "mpd", "space")
colnames(theta_guesses) <- c("unobs_sd", "invinc")

# Replace the zeros with NAs: BLPestimator will interpret this as no parameter
theta_guesses[theta_guesses == 0] <- NA

#Take a look!
theta_guesses
```

```
##       unobs_sd invinc
## price       NA    -50
## const        1     NA
## hpwt         1     NA
## air          1     NA
## mpd          1     NA
## space        1     NA
```

```r
car_est <- estimateBLP(
  blp_data = car_data,
  par_theta2 = theta_guesses,
  solver_method = "BFGS", solver_maxit = 1000, solver_reltol = 1e-9,
  standardError = "cluster",
  extremumCheck = FALSE, printLevel = 0
)
```

```
## blp_data were prepared with the following arguments:
## BLP_data(model = blps_model, market_identifier = "cdid", product_identifier = "id",
```

```
##     group_structure = "product_name", productData = productData_cars,
##     demographic_draws = demog_auto, integration_accuracy = ndraws,
##     integration_method = "MLHS", integration_seed = 48, blp_inner_tol = 1e-09,
##     blp_inner_maxit = 5000)
## ----------------------------------------
## Solver message: Successful convergence
## ----------------------------------------
## Final GMM evaluation at optimal parameters:
## gmm objective: 148.7971
##   theta (RC): 0 -3.45 -1.74 -0.93 0.94 4.6
##   theta (demogr.): -56.56 0 0 0 0 0
##   inner iterations: 263
##   gradient: -2e-04 -4e-04 -4e-04 0.0045 9e-04 1e-04
```

```r
# Take a look and compare with BLP(1995):

summary(car_est)
```

```
##
## Data information:
##
##   20 market(s) with 2217 products
##   5 linear coefficient(s) (5 exogenous coefficients)
##   6 non-linear parameters related to random coefficients
##   1 demographic variable(s)
##
## Estimation results:
##
##  Linear Coefficients
##         Estimate Std. Error   t value     Pr(>|t|)
## const -3.8815455  0.9604628 -4.041328 5.314934e-05
## hpwt   5.9199298  1.4836824  3.990025 6.606635e-05
## air    1.7354244  0.6401767  2.710852 6.711054e-03
## mpd   -0.3017639  0.2681686 -1.125277 2.604718e-01
## space  2.1109706  0.4220073  5.002214 5.667569e-07
##
##  Random Coefficients
##                   Estimate Std. Error    t value     Pr(>|t|)
## unobs_sd*const  -3.4503944  1.2433601 -2.7750565 5.519212e-03
## unobs_sd*hpwt   -1.7442410  1.7203844 -1.0138670 3.106462e-01
## unobs_sd*air    -0.9271268  1.8818646 -0.4926639 6.222501e-01
## unobs_sd*mpd     0.9397882  0.2939324  3.1972938 1.387236e-03
## unobs_sd*space   4.6008621  0.9201778  4.9999707 5.733904e-07
## invinc*price   -56.5557984 11.0458832 -5.1200793 3.054071e-07
##
##  Wald Test
## 44.9464 on  6 DF, p-value: 4.79639915771931e-08
##
## Computational Details:
##   Solver converged with 79 iterations to a minimum at 148.7971 .
##   Local minima check: NA
##   stopping criterion outer loop: 1e-09
##   stopping criterion inner loop: 1e-09
##   Market shares are integrated with MLHS and 500 draws.
##   Method for standard errors: cluster
```

I tried several starting values and the GMM criterion function of about 150 was the lowest I found, consistent with it being a global minimum. The linear estimates are mostly significantly different from zero with the exception of mpd. Most of the spread parameters are significantly different from zero. We can say that the average taste for mpd is quite close to zero according to the model, but that there is taste heterogeneity in the population for this attribute. The price parameter is significantly negative as we expect. This model's specification ensures that there are no consumers with a positive price coefficient, which is a desirable feature since price should reduce the indirect utility from a car. Note also the low constant term: this is a consequence of the definition of the outside good which is "not to buy a car" in a given market (i.e. year). Since most people don't buy a car in a given year, and the mean utility of this is scaled to zero, we expect the mean utility on an inside good to be negative.

Of course, it is difficult to interpret the parameters without looking to see the elasticities and implied marginal costs (given an assumption about pricing).

Point c)

```
# Get ownership matrix
own_pre <- dummies_cars
#own_pre <- as.matrix(car_data$data$additional_data[, paste0("company", 1:nmkt)])
```

```
# Store the new parameters from the estimates at point b)
theta_new <- matrix(NA, nrow = 6, ncol = 2)
rownames(theta_new) <-c("const", "hpwt", "air", "mpd", "space", "price")
colnames(theta_new) <- c("unobs_sd", "invinc")


theta_new[1,1] = car_est$theta_rc["unobs_sd*const"]
theta_new[2,1] = car_est$theta_rc["unobs_sd*hpwt"]
theta_new[3,1] = car_est$theta_rc["unobs_sd*air"]
theta_new[4,1] = car_est$theta_rc["unobs_sd*mpd"]
theta_new[5,1] = car_est$theta_rc["unobs_sd*space"]
theta_new[6,2] = car_est$theta_rc["invinc*price"]
theta_new
```

```
##         unobs_sd   invinc
## const -3.4503944       NA
## hpwt  -1.7442410       NA
## air   -0.9271268       NA
## mpd    0.9397882       NA
## space  4.6008621       NA
## price         NA -56.5558
```

```
# Get the firm market share
shareObj <- getShareInfo(
blp_data = car_data,
par_theta2 = theta_new,
printLevel = 2
)
```

```
## Mean utility (delta) is used as provided in the BLP_data() function.
```

```
# Define required elements

market_id <- productData_cars$cdid
nmkt <- length(unique(market_id))
  #car_data$parameters$market_id
```

```r
markups <- numeric(length(market_id))
sh <- shareObj$shares
prices_pre <- car_data$data$X_rand[, "price"]
```

```r
# Update mean utility
delta_pre <- car_est$delta
delta_data <- data.frame(
  "id" = car_data$parameters$product_id,
  "cdid" = car_data$parameters$market_id,
  "delta" = delta_pre
)
```

```r
# Update of car_data using the delta from data
car_data_updated <- update_BLP_data(
  data_update = delta_data,
  blp_data = car_data
)
```

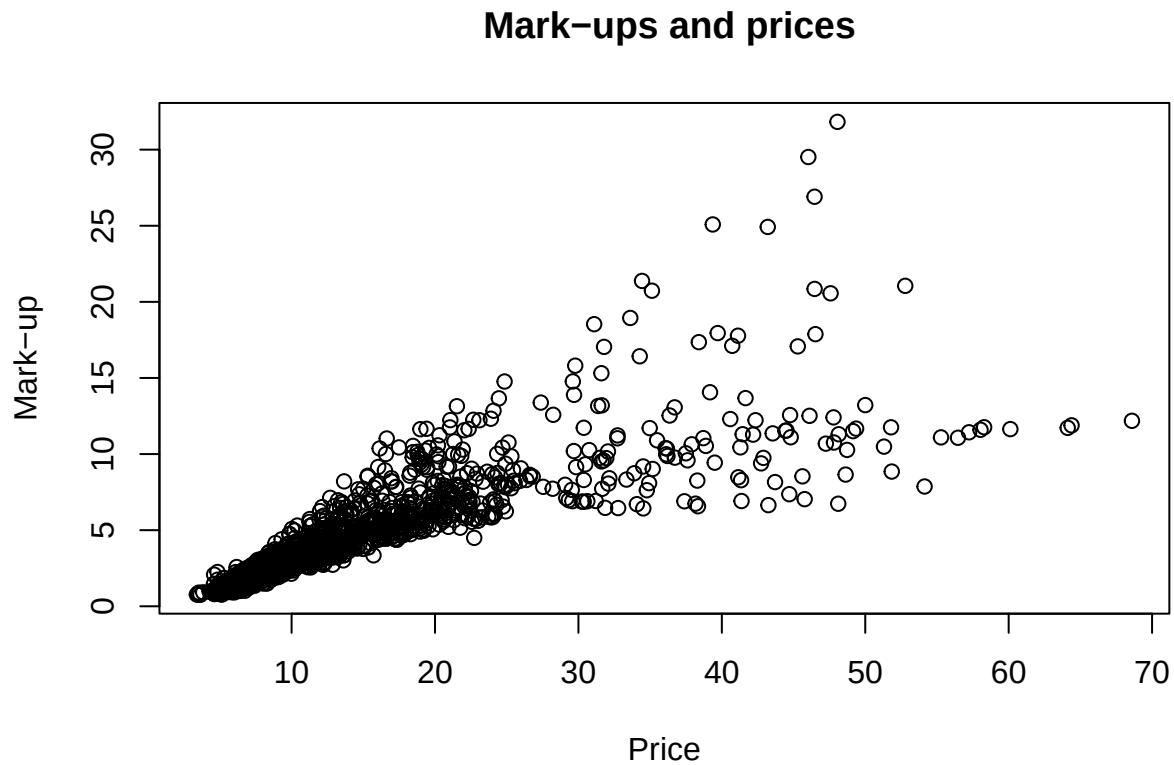
```
## Mean utility variable delta has been updated.
```

```r
# For loop for each market:
for (i in 1:nmkt) {
mkt_ind <- market_id == i
share_i <- sh[ mkt_ind ]
price_pre_i <- prices_pre[ mkt_ind ]
scalar_i <- matrix(1 / share_i) %*% matrix(price_pre_i, nrow = 1)

# Function to get elasticity_i
elasticities_i <- get_elasticities(
blp_data = car_data_updated,
share_info = shareObj,
theta_lin = 0,
variable = "price",
market = i,
printLevel = 0)

# partial derivatives of shares wrt price:
derivatives_i <- elasticities_i / scalar_i
own_pre_i <- own_pre[ mkt_ind, ]

# if element (i,j) equals 1, prod i and j are produced by same firm:
own_prod_pre_i <- own_pre_i %*% t(own_pre_i)

markups[mkt_ind] <- c(-solve( t(derivatives_i)*own_prod_pre_i) %*% share_i)
}

marg_cost <- prices_pre - markups
summary(marg_cost)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.519   4.975   6.167   8.146   8.528  56.411
```

We see that with multi-product Nash equilibrium pricing the marginal costs which are implied are positive, which is encouraging and much better than we saw with the simple logit model. This suggests the parameters are plausible in terms of the own- and cross-price elasticities.

```
plot(x = p, y = markups, main = " Mark-ups and prices", xlab = "Price", ylab = "Mark-up")
```



We see that higher prices tend to go hand in hand with higher markups. This is quite an interesting feature and suggests that high-price cars attract consumers with low alpha parameters (making it optimal for producers to set high markups).

We have broken free from the inflexibility associated with the simple logit model in which markups are given by the formula below. The simple logit model would not vary very much, because as we see from the below equation alpha does not vary (and because these market shares are all very small and hence vary in a very minimal way in the data):

$$p_{jt} - c_{jt} = -\frac{1}{\alpha(1 - s_{ft})}$$

Part d)

We now calculate a 5 ×5 matrix of own- and cross-price elasticities for some products.

```
# 5589 is a Yugo selling at $3,393
# 5478 is a Ford Fiesta at $4,834
# 5434 is a BMW 7 series at $25528
# 5447 is a Cadillac at  $40,589
# 5590 is a Porsche 911 selling at $44,759


#Calculate elasticities for selected products
# Order them from cheapest to most expensive:
```

```r
Elas_autos_v2<-get_elasticities(
  blp_data = car_data_updated,
  share_info = shareObj,
  theta_lin = 0,
  variable = "price",
  products = c("5589", "5478", "5428", "5447", "5590"),
  market = "20"
)


#Label columns for ease of reading
colnames(Elas_autos_v2) <- c("Yugo", "Fiesta",   "BMW5", "Cadillac", "Porsche")

# effect of change in column price on row share
Elas_autos_v2
```

```
##              Yugo       Fiesta          BMW5      Cadillac       Porsche
## 5589 -4.379087237  0.164535172  2.568429e-05  3.832461e-06  3.260448e-06
## 5478  0.431377632 -4.143743550  7.613777e-05  1.222412e-05  1.390526e-05
## 5428  0.002732747  0.003089819 -3.202561e+00  2.636495e-03  5.089825e-04
## 5447  0.001297078  0.001578001  8.386552e-03 -3.691593e+00  5.737164e-04
## 5590  0.002880183  0.004685146  4.225847e-03  1.497448e-03 -3.563951e+00
```

We have picked the products such that we expect the first two products attract price-sensitive consumers and are good substitutes for each other, and the last 3 products attract price-insensitive consumers and also are close substutes between the three of them.

The own-elasticities, which are located on the main diagonal, are all smaller than -1 (so, they are plausible). Also, the demand for the cheap cars is more elastic than for any of the others which is quite plausible and the opposite of what is implied by the simple logit where elasticity tends to be greater for high-priced products relative to low-priced products (when market shares are similar)

The cross-elasticities are on the off-diagonals. The cell (column i, row j) give the cross elasticity of the product in row j with respect to the price in column i. We see that when the low-price cars increase price, the cross elasticity between the is much higher than for the other three products. For example, for the Yugo, the elasticity of demand for a Fiesta with respect to the price of a Yugo is 0.43 which is much higher than the elasticity of demand for a BMW5 wrt the price of a Yugo (which is 0.002). This again breaks the inflexibility of the simple logit model in which all cells in a given column would be identical off the diagonal.

Part e)

```r
# Merger between company 16 and 19
# New ownership matrix

own_post <- cbind(
  own_pre[, 1:15],
  own_pre[, 16] + own_pre[, 19],
  own_pre[, 17:18],
  own_pre[, 20:26]
)
```

```r
foc_bertrand_mkt <- function(par, own_prod, blp_data, mkt,
marg_cost, theta_lin, theta_rc) {

tmp <- data.frame("id" = blp_data$parameters$product_id,
```

```r
"cdid" = blp_data$parameters$market_id,
"price" = blp_data$data$X_rand[, "price"] )
t_ind <- blp_data$parameters$market_id == mkt
prices_pre <- blp_data$data$X_rand[, "price"]
tmp$price[ t_ind ] <- par

new_blp_data <- update_BLP_data( blp_data = blp_data, data_update = tmp )

ShareObj <- getShareInfo( blp_data = new_blp_data,
par_theta2 = theta_rc, printLevel = 0)

implied_shares <- as.matrix(ShareObj$shares[mkt_ind])
elasticities_post_mkt <- get_elasticities( blp_data = new_blp_data,
share_info = ShareObj,
theta_lin = 0,
variable = "price",
market = mkt,
printLevel = 0)

scalar_mkt <- matrix(1 / implied_shares) %*% matrix(par, nrow = 1)
derivatives_mkt <- elasticities_post_mkt / scalar_mkt

markups_post <- c(-solve(t(derivatives_mkt)*own_prod) %*% implied_shares)
differences <- par - marg_cost[mkt_ind] - markups_post
return(differences)
}
```

```r
#Pick market 2
#Pick a market (i can be anything between 1 and 20)
i=2
  mkt_ind <- market_id == i
  own_post_i <- own_post[ mkt_ind, ]
  own_prod_post_i <- own_post_i %*% t(own_post_i)
  price_pre_i <- prices_pre[ mkt_ind ]

  solution <- nleqslv(
    x = price_pre_i, foc_bertrand_mkt, # startingguesses: price_pre_i
    own_prod = own_prod_post_i,
    blp_data = car_data_updated,
    mkt = i,
    marg_cost = marg_cost,
    theta_lin = 0,
    theta_rc = theta_new
  )
```

```
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
```

```
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
```

```
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
## Random coefficient variable price has been updated.
```

```
  #prices_post[ market_id == i ] <- solution$x
  prices_post_i <- solution$x


pch = (prices_post_i - price_pre_i)/price_pre_i
```

```
mean(pch < 0) # Proportion of firms lowering their prices post-merger
```

```
## [1] 0.5505618
```

```
summary(pch)
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -0.127116 -0.033739 -0.007011  0.014554  0.063891  0.159067
```

```
cars_new <- productData_cars[mkt_ind==1,]

summary(pch[cars_new$firmid==16|cars_new$firmid==19]) # Summary for merging firms
```

```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## -0.05547  0.04020  0.06686  0.06681  0.12400  0.15907
```

```r
summary(pch[!(cars_new$firmid==16|cars_new$firmid==19)]) # Summary for non-merging firms
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## -0.127116 -0.040219 -0.025416 -0.030076 -0.008658  0.016023
```

Comment: it took some attempts to get an example which would solve. This is a merger for firms 16 and 19 in the second market (1971). The results are a little odd in that the median price change is negative from the merger which is inconsistent with what we would expect. Further work is needed (but outside the scope of the exercise) to investigate whether this is because there are multiple equilibria in multi-product Nash pricing or if its an issue with the equation solver. We can also see that most of the products of the merging firms see a price increase, which is consistent with what we would expect.

END.